

PRD — ValidationKit *(Working Title)*

Open-Source-Framework zur AI-gestützten Validierung von Geschäftsideen. Primär als Claude-Code-Subagent-System aufgebaut. Web-Layer kommt später als Open-Core-Monetarisierung.

0. Meta

Status	Draft v0.1 — als Basis für Iteration in Claude Code
Erstellt	2026-05-13
Owner	[Du]
Distribution	Open-Source-Framework (MIT) + späterer SaaS-Layer
Primärer Build-Stack	Claude Code + Claude Agent SDK

Zweck dieses Dokuments: Grundlage für das Weiter-Iterieren in Claude Code. Wo Fragen offen sind, ist das markiert mit > [OFFEN] .

1. Vision

Solopreneurs und technische Founder verbrennen Monate, indem sie Produkte bauen, die niemand will. Existierende AI-Validatoren liefern generische 120-Sekunden-Scores ohne actionable Output. Echte User-Interview-Plattformen sind teuer und auf Enterprise zugeschnitten.

ValidationKit ist das **Framework**, das den kompletten Validation-Loop in deiner Claude-Code-Umgebung orchestriert: von der Idee über Markt-Recherche, Persona-Simulation, legitime Outreach-Drafts, Fake-Door-Tests bis zur Pivot-Empfehlung — alles als komponierbare Subagents, die du selbst forken und erweitern kannst.

Tagline-Kandidaten:

- "Validate before you build."

- "Demand-first development for solo founders."
 - "Your AI agent team for product-market fit."
-

2. Problem Statement

Drei zusammenhängende Probleme:

1. **AI-Auto-Validatoren produzieren Brainstorming-Output, keine Entscheidungs-Daten.** Score 87/100 sagt nichts darüber, ob jemand zahlen würde.
2. **Echte Validation (Interviews, Surveys) ist teuer, langsam und nicht im Solo-Founder-Budget** — User Intuition \$20/Interview, Wynter \$499/Test.
3. **Niemand orchestriert den vollen Stack:** Synthetic Personas (für Speed) → echte Outreach an legitime Channels (für Trust-Signale) → echte Conversion-Tests (Fake Door, Ads) → Synthese in actionable Pivot-Empfehlung.

Bisherige Tools sind Punkt-Lösungen (Delve AI = nur Personas, IdeaProof = nur Report, GoZigzag = monolithisch und Closed-Source).

3. Zielgruppen & Personas

Primär (P0)

Technische Solopreneurs / Indie Hackers

- Bereits in Claude Code / Cursor / Codex aktiv
- Wollen Tooling, keine SaaS-UI
- Budget: \$0–\$50/Monat, aber tief technisch versiert
- Bauen meist B2B SaaS, Dev Tools oder AI-Wrapper

Sekundär (P1, erst nach MVP)

Boutique-Agenturen & Consultants

- Validieren Ideen *für* Kunden
- Wollen White-Label-Reports + Repeatability
- Budget: \$200–\$1000/Monat

Explizit nicht Zielgruppe (P-never für MVP)

- Enterprise Innovation Teams (zu lange Sales-Cycles, falsche DNA)
- Non-technical Solopreneurs (für die ist die UI-First-Konkurrenz wie GoZigzag/IdeaProof besser)

4. Wettbewerbslandschaft (Stand 2026)

Kategorie	Beispiele	Stärke	Schwäche
AI Auto-Validators	IdeaProof, ValidatorAI, DimeADozen, GoZigzag, PrometAI	Schnell, breit	Generisch, keine echten User-Signale
AI-Moderated Interviews	User Intuition, Wynter	Echte Daten	Teuer, langsam, Enterprise-Bias
Synthetic Personas	Delve AI, sampl.space, PyMC Labs	Skalierbar	Nur Vorstufe, keine Conversion-Tests
Outreach-Automation	Expandi, HeyReach, Waalaxy, ColdDMs	Skalierung	ToS-Verstoß, Account-Risiko
Dev-Frameworks	wshobson/agents, awesome-claude-code-subagents	Composable	Generisch, kein Validation-Focus

Direkteste Konkurrenz: GoZigzag (macht ähnlichen End-to-End-Loop, aber Closed-Source SaaS). **Konkrete Diff:** Open-Source, Dev-First, Composable, legit channels only, kein "Logo/Brand"-Bloat.

5. Value Proposition & Differenzierung

Was ValidationKit anders macht

1. **Composable statt monolithisch** — jeder Subagent ist ein eigenes File, das du replacen kannst
2. **Real Signals statt Vibe-Score** — fake-door tests, paid-ad spend tests, cold-email reply rates als First-Class-Outputs

3. **Synthetic Personas als Pre-Filter** — nicht als Ersatz für echte User
4. **Legitimate Channels only** — keine DM-Automation, keine ToS-Verstöße, Founder behält Account-Reputation
5. **Open-Source-Core** — du kannst forken, anpassen, in CI/CD einbinden

Was es explizit *nicht* ist

- Kein automatisiertes Spam-Tool für Instagram/LinkedIn-DMs
 - Kein "Brand Identity + Logo + Pitch Deck"-Generator (abgegrast bei IdeaProof & Co)
 - Kein Replacement für echte Customer Interviews bei >\$10k-Entscheidungen
-

6. MVP-Scope (Phase 0)

Build-Ziel: In 4–6 Wochen ein installierbares Claude-Code-Framework, das eine Idee durchläuft und einen Validation-Report + erste konkrete Action-Items produziert.

Phase-0-Features

- Installer: `npx create-validationkit@latest` legt `.claude/`-Struktur an
- **8 Core Subagents** (siehe §7)
- **5 Slash Commands** (siehe §8)
- **Validation Report** als strukturiertes Markdown-Output
- **Persistent State** in `.validationkit/state.json` (welche Ideen, welche Phase)
- CLI-only — keine Web-UI in Phase 0
- Eine "Reference Implementation" mit einer echten validierten Idee als Beispiel

Phase-0-Erfolgskriterien

- Ein User kann von "Idee in Prompt" zu "Validation Report + Cold-Email-Draft + Fake-Door-Page-Spec" in <30 Minuten kommen
 - Mindestens 3 echte Indie Hackers nutzen das Framework end-to-end und geben Feedback
 - GitHub: 100+ Stars in den ersten 30 Tagen post-Launch
-

7. Framework-Architektur

```
validationkit/
├── .claude/
│   ├── agents/                                # Subagents (8 in Phase 0)
│   │   ├── idea-clarifier.md
│   │   ├── market-researcher.md
│   │   ├── persona-generator.md
│   │   ├── persona-interviewer.md
│   │   ├── channel-strategist.md
│   │   ├── outreach-writer.md
│   │   ├── fake-door-designer.md
│   │   └── feedback-synthesizer.md
│   ├── commands/                             # Slash Commands
│   │   ├── validate.md
│   │   ├── persona-test.md
│   │   ├── landing-page.md
│   │   ├── outreach.md
│   │   └── synthesize.md
│   ├── skills/                               # Reusable Skills
│   │   ├── problem-discovery/
│   │   ├── competitor-analysis/
│   │   └── cold-email-craft/
│   └── settings.json
├── .validationkit/
│   ├── state.json                            # Aktuelle Idee + Phase
│   └── reports/                             # Generierte Reports
└── README.md
```

Subagent-Orchestrierung

Hauptagent (Claude Code Session) orchestriert die Subagents in einer **klaren Pipeline**, aber jede Phase kann standalone laufen:

```
[Idee] → idea-clarifier → market-researcher → persona-generator
      ↓
persona-interviewer (parallel: 3-5 Personas)
      ↓
channel-strategist → outreach-writer + fake-door-designer (parallel)
      ↓
[Mensch führt aus: Posts, Emails, Landing Page Live]
      ↓
feedback-synthesizer → Validation Report + Pivot/Go-Empfehlung
```

8. Core Subagents (Phase 0)

Jeder Subagent ist ein Markdown-File mit YAML-Frontmatter (Claude-Code-Konvention).

8.1 `idea-clarifier`

Trigger: Initial-Start, vage Idee **Tools:** Read, Write, AskUser **Job:** Stellt 5–10 Klärungsfragen (Wer hat das Problem? Wie oft? Was wäre Pain ohne Lösung? Existierende Workarounds?) und produziert eine strukturierte Problem-Statement-Datei.

8.2 `market-researcher`

Tools: WebSearch, WebFetch, Read, Write **Job:** Sucht 5–10 direkte/indirekte Konkurrenten, sammelt Pricing, USPs, Reviews. Output: `competitive-landscape.md`.

8.3 `persona-generator`

Tools: Read, Write **Job:** Generiert 3–5 detaillierte Buyer Personas (Demografie, Jobs-to-be-Done, aktuelle Workarounds, Budget, Decision-Trigger). Mindestens 1 Skeptiker, 1 Early Adopter, 1 Mainstream.

8.4 `persona-interviewer`

Tools: Read, Write, Task (spawnt Sub-Sub-Agents) **Job:** Spawnt für jede Persona einen Sub-Agent, der ein simuliertes Interview führt. Sammelt: Pain-Severity-Score, Will-Pay-Indikator, Bedenken, Alternative-Lösungen.

8.5 `channel-strategist`

Tools: Read, Write **Job:** Bewertet basierend auf Persona-Profilen, welche **legitimen** Channels Sinn machen: Cold Email, Reddit-Subs, IndieHackers, X, HN, Communities, Paid Ads. **Explizit ausgeschlossen:** LinkedIn-DM-Auto, Instagram-DM-Auto.

8.6 `outreach-writer`

Tools: Read, Write **Job:** Generiert pro empfohlenem Channel einen Outreach-Draft (Cold Email, Reddit Post, X Thread, etc.). User postet/schickt manuell.

8.7 `fake-door-designer`

Tools: Read, Write **Job:** Produziert Landing-Page-Copy + Wireframe-Spec für Fake-Door-Test (Headline, Subhead, CTA, "Join Waitlist"-Flow). Optional: HTML-Output für Vercel/Netlify-Deploy.

8.8 feedback-synthesizer

Tools: Read, Write, WebFetch **Job:** Nimmt echte Daten (Email-Replies, Landing-Page-Conversions, Ad-CTRs, Reddit-Comments) und destilliert in: Demand-Signal-Score, Top-3 Insights, Pivot-Empfehlung oder Go-to-Build.

9. Slash Commands

Command	Subagents involved	Output
<code>/validate "<idea>"</code>	clarifier → researcher → personas → interviews	<code>reports/initial-validation.md</code>
<code>/persona-test</code>	persona-interviewer	Persona-Interview-Transkripte
<code>/outreach <channel></code>	channel-strategist → outreach-writer	Drafts in <code>drafts/</code>
<code>/landing-page</code>	fake-door-designer	HTML + Copy in <code>landing/</code>
<code>/synthesize <data-folder></code>	feedback-synthesizer	Final Report + Go/Pivot/Kill

10. Tech Stack

Layer	Wahl	Begründung
Agent Runtime	Claude Code + Claude Agent SDK	User ist schon hier, Subagents nativ
Sprache	TypeScript (Tooling) + Markdown (Agents)	Standard im Claude-Code-Ökosystem
MCP Integrations (Phase 0)	WebSearch, WebFetch	Sind built-in
MCP Integrations (Phase 1)	Resend (Email), Vercel (Deploy), Reddit/X-API (Search)	Optional, opt-in

State	Lokales JSON in <code>.validationkit/</code>	Keine Cloud-Dependency Phase 0
Distribution	npm + GitHub	Standard, Discoverable

[OFFEN] TypeScript vs Python? Wenn Web-Layer später Next.js → TS einheitlich. Wenn ML-Heavy später → Python. **Empfehlung: TS.**

11. Non-Goals (Explizit ausgeschlossen)

- **✗ Automatisierte LinkedIn/Instagram-DMs** — ToS-Verstoß, Account-Risiko, falsches Signal
- **✗ Logo-/Brand-Identity-Generator** (Commodity, schlecht zu differenzieren)
- **✗ Pitch-Deck-Generator** (abgegrast, nicht im Validation-Loop)
- **✗ Vollautomatisches "Idee → Live-Produkt"-Pipeline** (zu komplex, zu fehleranfällig)
- **✗ Eigene LLM-Hosting** (nutze Claude via SDK)
- **✗ Enterprise-Features** (SSO, Audit-Logs, Multi-Tenant) in Phase 0

12. Roadmap

Phase 0 — MVP (Wochen 1–6)

Framework + 8 Subagents + 5 Commands. CLI-only. GitHub-Launch.

Phase 1 — Real-Channel-Integration (Wochen 7–12)

- Resend-MCP für Cold-Email-Send (mit Consent-Check)
- Vercel-MCP für Landing-Page-Deploy
- Reddit/HN/X-API für echtes Community-Listening
- Plausible/PostHog für Conversion-Tracking

Phase 2 — Web-Layer Beta (Monate 4–6)

- Next.js Dashboard für Non-Tech-User
- Auth + Multi-Project

- Hosted Pipelines (Framework läuft als Service)
- **Erste Monetization:** \$29/Monat Pro Plan

Phase 3 — Marketplace (Monate 7–12)

- Custom Persona-Packs (Industry-spezifisch)
 - Custom Agent-Templates
 - White-Label für Agenturen
 - \$99–299/Monat Team Plan
-

13. Monetization (Vorläufig)

Open-Core-Modell:

- **Free Forever:** Framework, alle Core Subagents, CLI
- **Pro (\$29/mo):** Web-UI, Hosted Pipelines, Email/Ad-Infra, längere History
- **Team (\$99/mo):** Multi-User, White-Label, Custom-Branded Reports
- **Marketplace-Revenue-Share:** 70/30 mit Persona-Pack-Creators

[OFFEN] Alternativ: Usage-based (\$/Validation-Run inkl. Email-Sends, Ad-Spend, etc.).

14. Success Metrics

Framework (Phase 0)

- GitHub Stars: 100 in 30 Tagen, 1000 in 6 Monaten
- npm Installs: 500/Monat nach 3 Monaten
- Aktive End-to-End-Validations: 50/Monat nach 3 Monaten
- Discord/Community: 200 Mitglieder

Web-Layer (Phase 2)

- Free-to-Pro-Conversion: 5–8%
- MRR nach Monat 9: \$5k

- Retention M3: >50%

Wirkungs-Metrik (Nordstern)

"X% der ValidationKit-User entscheiden sich gegen ihre ursprüngliche Idee oder pivotieren bevor sie programmieren." **Target:** ≥30% (das ist der eigentliche Wert.)

15. Risks & Mitigations

Risiko	Wahrscheinlichkeit	Impact	Mitigation
GoZigzag oder IdeaProof launchen Open-Source-Pendant	Mittel	Hoch	Speed: 6 Wochen zum MVP, Community-Building
Synthetic Personas werden als nutzlos wahrgenommen	Mittel	Mittel	Klare Positionierung als Pre-Filter, nicht Ersatz
Claude Code Subagent-Spec ändert sich	Niedrig	Hoch	Agent-SDK statt nur Markdown-Spec, Abstraktion eingebaut
Cold-Email-Send via Resend → Deliverability-Probleme	Mittel	Mittel	Opt-in pflicht, kleine Volumes, Best-Practice-Templates
User installieren, nutzen einmal, kommen nicht zurück	Hoch	Hoch	Wöchentlicher Validation-Loop, Re-Run mit neuen Daten

16. Open Questions

1. **Naming.** "ValidationKit" generisch. Alternativen: ProofStack, DemandLab, SignalKit, Validar, PMFKit. → Domain-Check + Trademark.
2. **TypeScript vs Python** für die Tooling-Layer (siehe §10).
3. **Reichen Synthetic Personas in Phase 0**, oder muss Cold-Email/Real-Channel-Integration schon im MVP sein? — Trade-off: 6 vs 10 Wochen.

4. **Lizenz:** MIT vs AGPL? AGPL würde verhindern, dass GoZigzag das forkt und Closed-Source-deployed.
 5. **Wie viel "AI Brand"** — eigene Domain & Identity oder als Plugin im wshobson/agents-Ökosystem launchen für initiale Distribution?
 6. **Erste 10 Design-Partner:** Wer? Wo aufschlagen — IndieHackers, Reddit r/SaaS, Twitter, Anthropic Discord?
-

17. Nächste Schritte (für deine nächste Claude-Code-Session)

Empfohlene Reihenfolge:

1. **Iteriere dieses PRD** — beantworte die §16 Open Questions
 2. **Validiere das Framework selbst mit dem Framework** ("Dogfooding"): nutze die manuellen Versionen der 8 Subagents, um ValidationKit gegen 5 echte Indie Hackers zu validieren
 3. **Repo-Setup:** GitHub-Org, License, README, CONTRIBUTING
 4. **Bau einen einzelnen Subagent** (idea-clarifier) end-to-end als Reference Implementation
 5. **Dann erst:** Rest der 8 Subagents parallel
 6. **Launch-Plan:** Soft Launch in Anthropic Discord + r/ClaudeAI + IndieHackers, dann breit
-

Ende PRD v0.1. — Iteriere frei.